Roma Tre

Information Visualization

Drawing Hierarchical Graphs

Part II

M. Patrignani

 copyright©2026 m.patrignani

Copyright notice

- All the pages/slides in this presentation, including but not limited to, images, photos, animations, videos, sounds, music, and text (hereby referred to as “material”) are protected by copyright
- This material, with the exception of some multimedia elements licensed by other organizations, is property of the authors and/or organizations appearing in the first slide
- This material, or its parts, can be reproduced and used for didactical purposes within universities and schools, provided that this happens for non-profit purposes
- Any other use is prohibited, unless explicitly authorized by the authors on the basis of an explicit agreement
- This copyright notice must always be redistributed together with the material, or its portions

 copyright©2026 m.patrignani

Summary

- Step 3 of the Sugiyama approach : Crossing reduction
  - Layer-by-layer sweep
  - The two-layer crossing problem
  - Polynomial-time heuristics
- Step 4: X-coordinate assignment

 copyright©2026 m.patrignani

Step 3: Crossing reduction

 copyright©2026 m.patrignani

Problem definition

Input	A proper layering of a directed acyclic graph G
Output	An ordering for the nodes of each layer
Target	Reduce the number of crossings between edges in order to improve the readability of the graph

 copyright©2026 m.patrignani

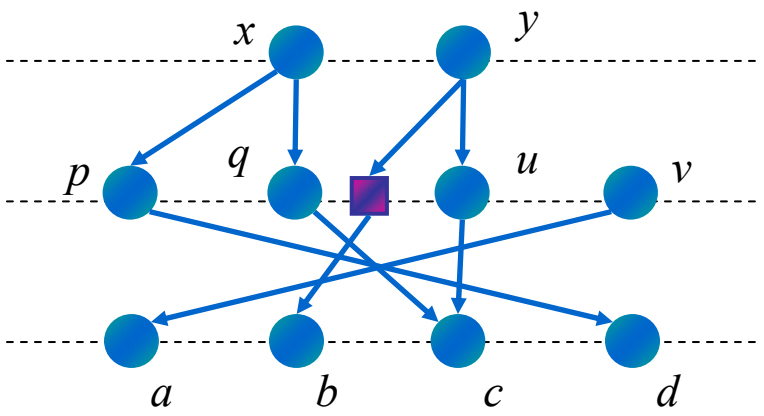
Preliminary observation

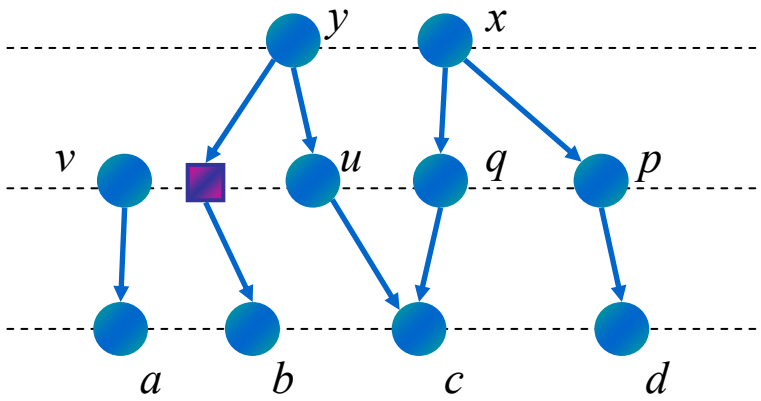
- The number of crossings
  - does not depend on the geometric coordinates of nodes
  - only depends on the order of the nodes of each layer
- Reducing the number of crossings is a combinatorial and not a geometric problem
  - we only have to choose a suitable order of nodes for each layer

 copyright©2026 m.patrignani

Example

- The number of crossings only depends on the order of the nodes on the layers
  - the orders on the left imply an high number of crossings
  - the orders on the right imply zero crossing





 copyright©2026 m.patrignani

Layer-by-layer Sweep and the Two-layer Crossing Problem

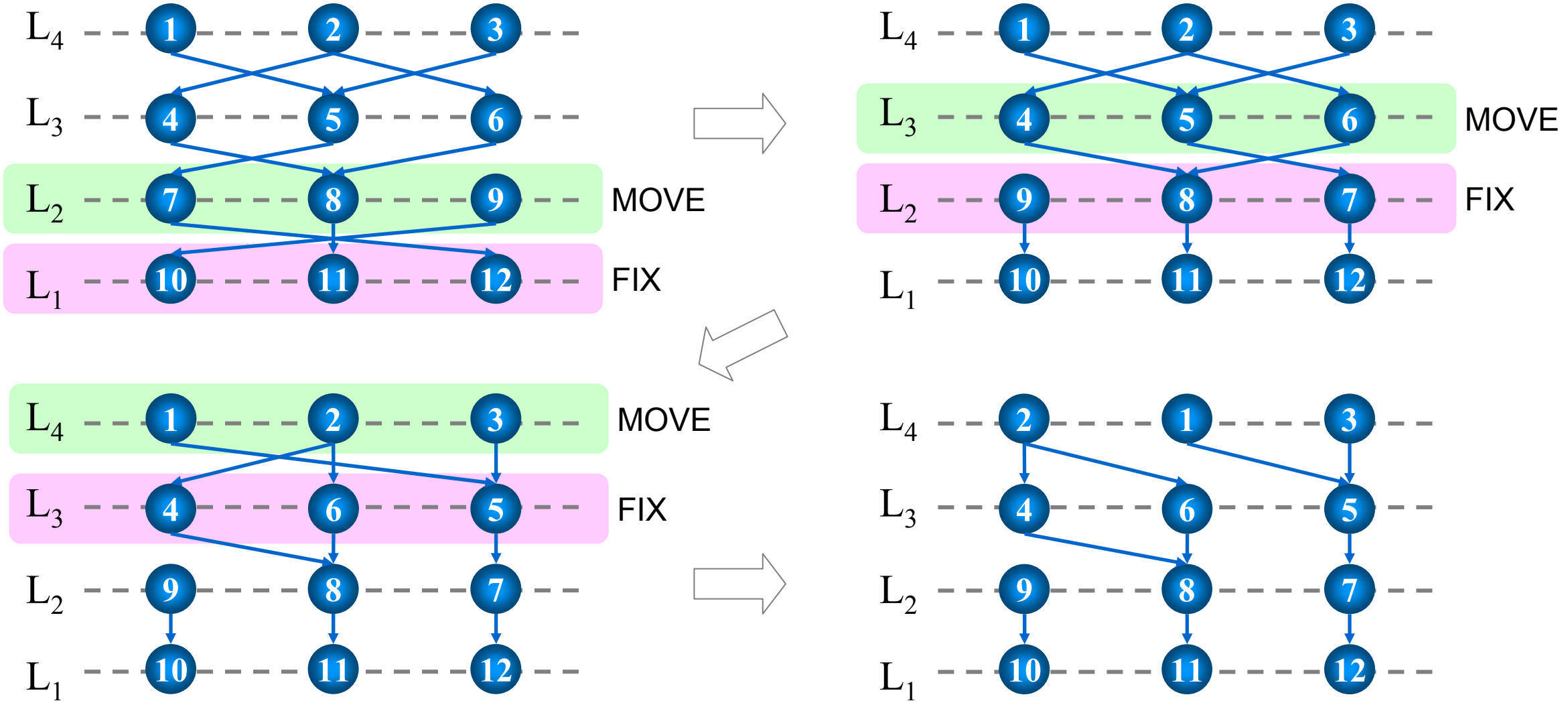
 copyright©2026 m.patrignani

Layer-by-layer sweep

- The layer-by-layer sweep is a general paradigm adopted by most crossing reduction techniques
  - first fix an ordering for layer L_1
  - then, for i = 2, 3, …, h, choose an order of the nodes of layer L_i in such a way to reduce the number of crossings between layers i and i-1
- Several variants are possible
  - for example: it is possible to fix the orderings of the nodes of both layers L_{i-1} and L_{i+1} while choosing the order of layer L_i

 copyright©2026 m.patrignani

Example



 copyright©2026 m.patrignani

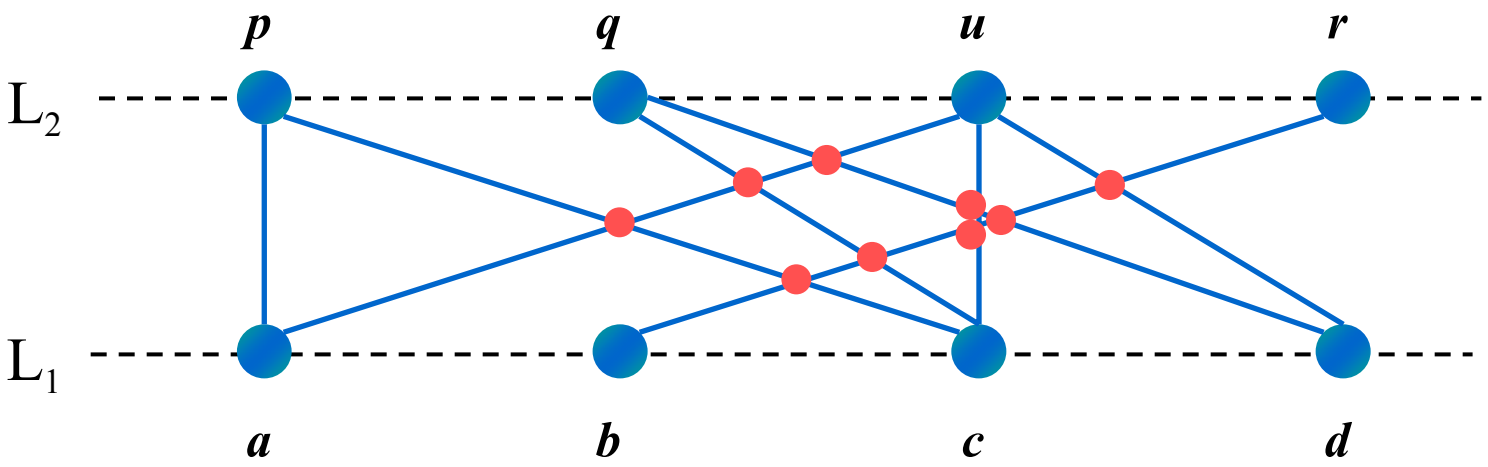
Two-layer crossing problem

- The layer-by-layer sweep approach is based on the solution of the following subproblem, known as two-layer crossing problem

Input	A layered graph G on two layers, where the ordering for the nodes of layer L_1 is fixed
Output	An ordering for the nodes of layer L_2
Target	Reduce the number of crossings between edges among the two layers

 copyright©2026 m.patrignani

Example



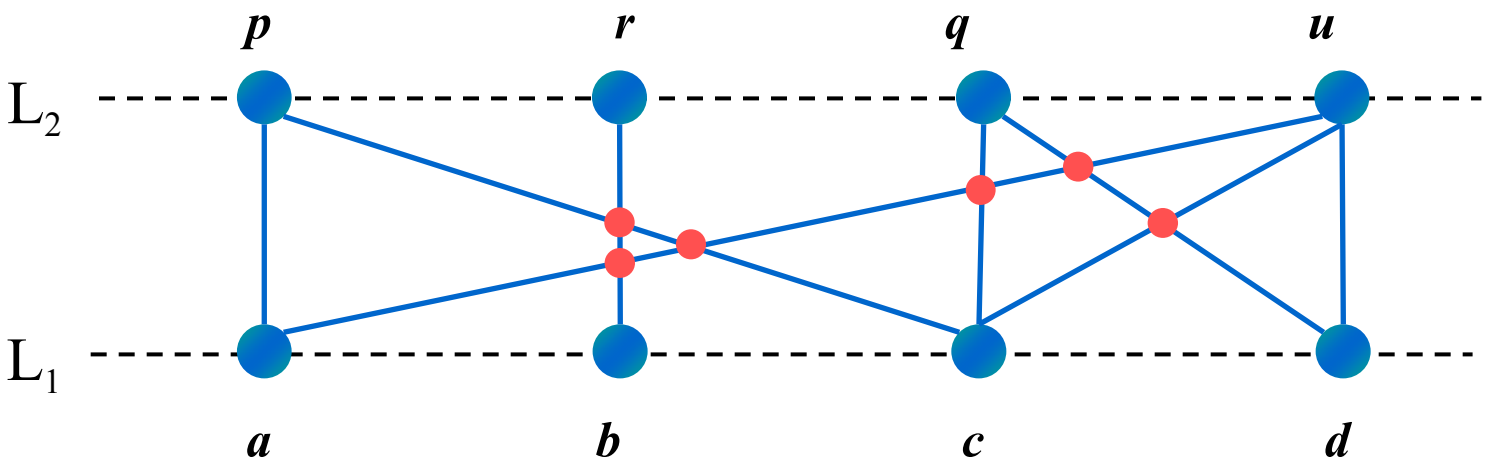
INPUT $\pi_1 = \{a, b, c, d\}$

OUTPUT $\pi_2 = \{p, q, u, r\}$

$\Rightarrow$ *9 crossings*

copyright©2026 m.patrignani

Example



INPUT $\pi_1 = \{a, b, c, d\}$

OUTPUT $\pi_2 = \{p, r, q, u\}$

$\Rightarrow$ *6 crossings*

copyright©2026 m.patrignani

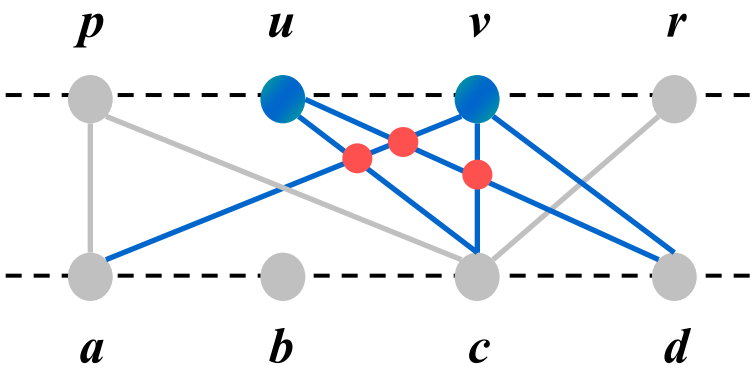
Problem complexity

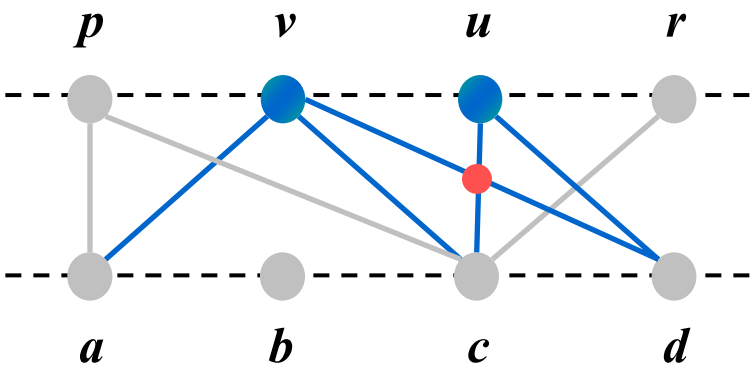
- The two-layer crossing problem is NP-complete
  - [Eades & Whitesides, 94]
- If you also allow to change π_1 the problem is also NP-complete
  - [Shaefer 13]
- Possible approach are
  - polynomial-time heuristics
  - approximated algorithms
  - exponential-time exact algorithms
  - FPT algorithms, etc

 copyright©2026 m.patrignani

Preliminary concepts

- Let u and v be two nodes of L_2
- The number of crossings among edges incident to u and v only depends on the relative positions of u and v
  - it does not depend of the positions of other nodes

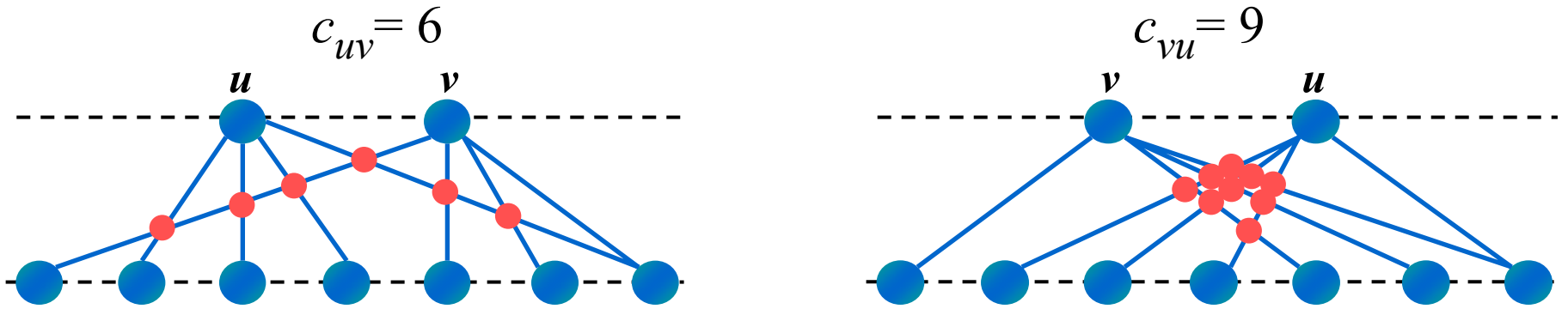




 copyright©2026 m.patrignani

Pair crossing number

- Let *u* and *v* be two nodes of L_2
- The *pair crossing number* c_{uv} is the number of crossings among edges outgoing *u* and edges outgoing *v*, whenever *u* is placed before *v* by π_2, that is, $\pi_2(u) < \pi_2(v)$
  - in general c_{uv} is different from c_{vu}



copyright©2026 m.patrignani

Using pair crossing numbers

- Pair crossing numbers may be useful when devising heuristics for the two-layer crossing problem
  - in fact $\text{cross}\ (G, \pi_1, \pi_2) = \sum_{\pi_2(u) < \pi_2(v)} c_{uv}$
- Pair crossing numbers offer a lower bound for the two-layer crossing problem
  - in fact $\min_{\pi_2}(\text{cross}(G, \pi_1, \pi_2)) \geq \sum_{u,v \in L_2} \min(c_{uv}, c_{vu})$

copyright©2026 m.patrignani

Polynomial-time heuristics

- Adjacent-exchange
  - similar to bubble-sort
- Split heuristic
  - similar to quick-sort
- Baricentric heuristic
- Median heuristic

 copyright©2026 m.patrignani

Adjacent-exchange

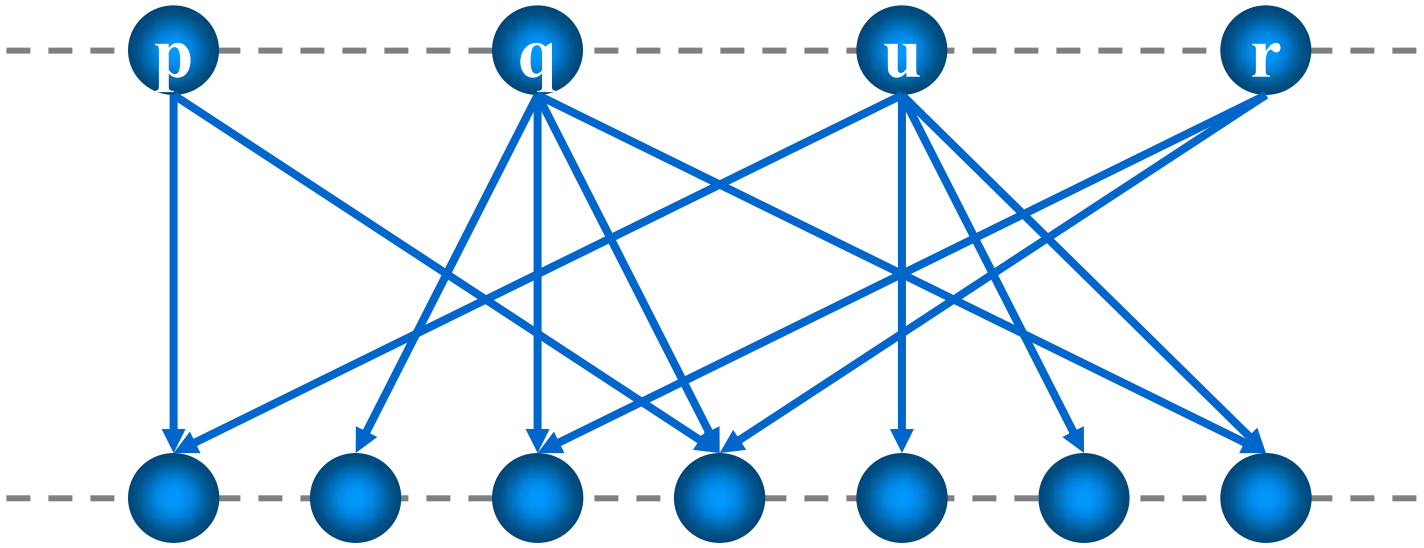
- Swap pairs of adjacent nodes if their crossing number decreases
  - approach very similar to bubble sort
  - introduced by [Eades & Kelly, 86]

```
Adjacent_exchange
/* Input: a two-layered digraph G = (L1, L2, E) and
            an ordering π1 of nodes of L1*/
/* Output: an ordering π2 of nodes of L2 */

    1. Compute c_uv and c_vu for any pair u, v
    2. Choose one initial ordering for L2
    3.  Repeat
            Sweep nodes of L2 from left to right,
            swapping   adjacent pairs u, v if c_uv > c_vu
        Until  no more swaps are possible
```

 copyright©2026 m.patrignani

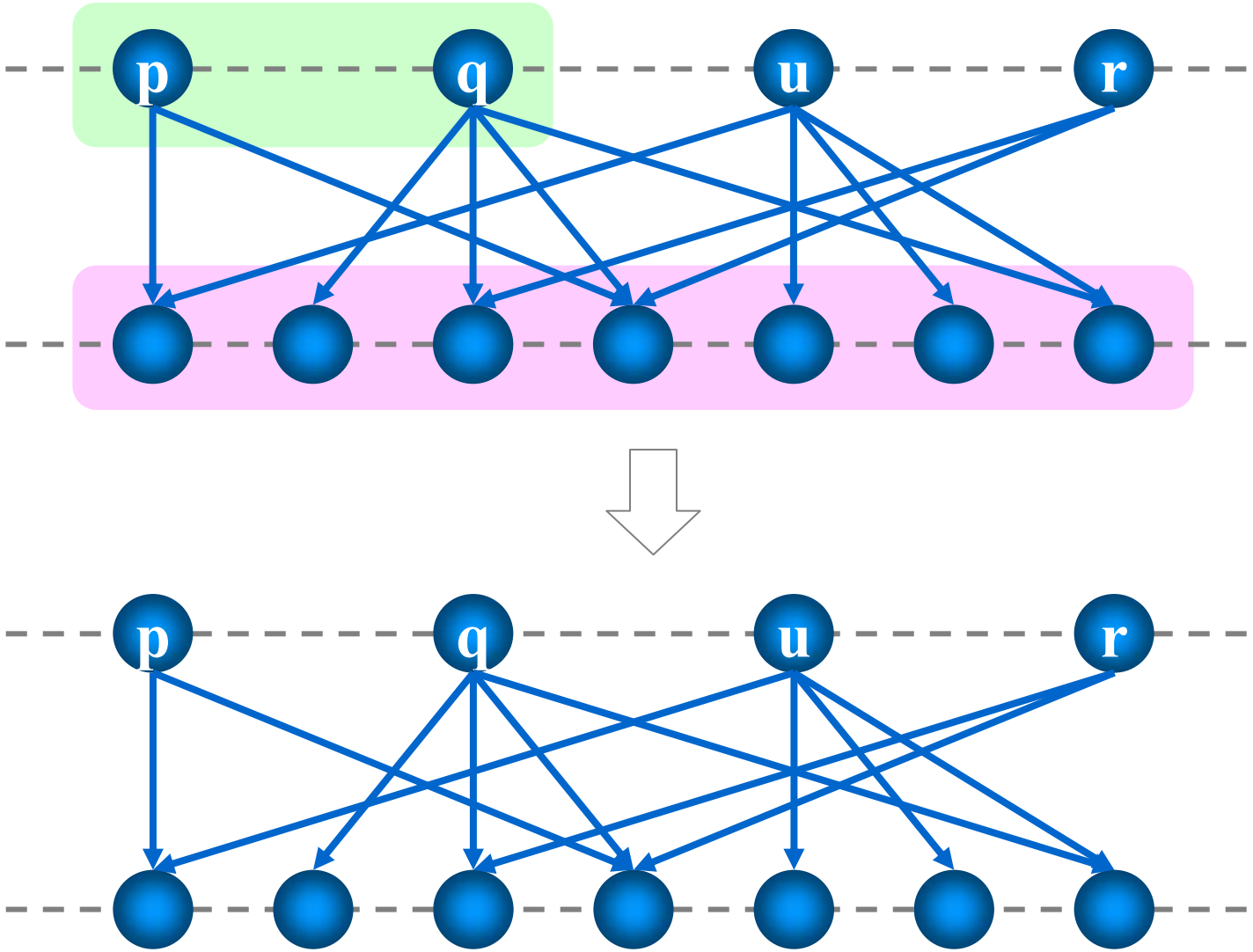
Adjacent-exchange: example



	p	*q*	*u*	*r*
p	0	2	1	1
q	5	0	6	3
u	6	9	0	6
r	2	3	2	0

copyright©2026 m.patrignani

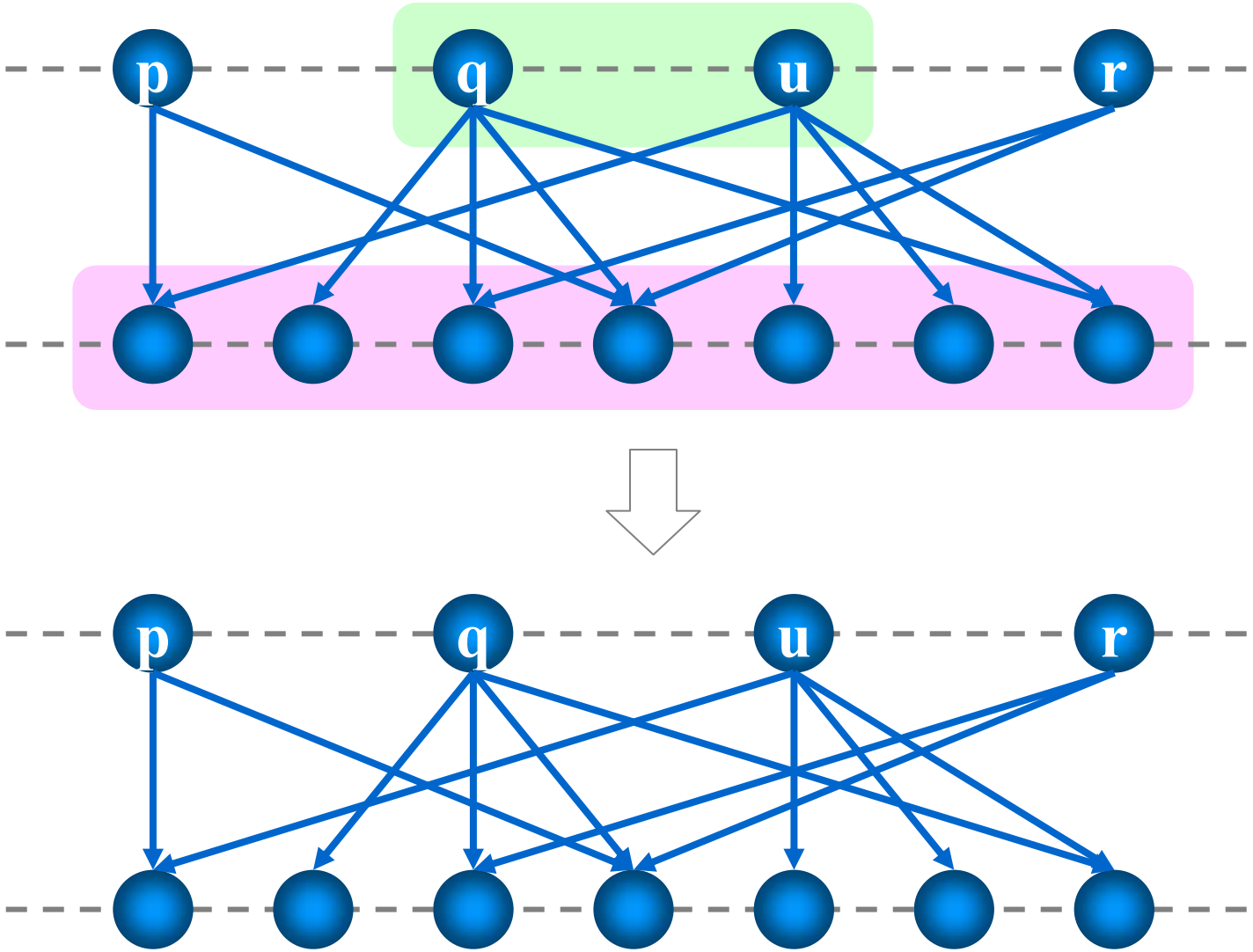
Adjacent-exchange: example



	p	*q*	*u*	*r*
p	0	2	1	1
q	5	0	6	3
u	6	9	0	6
r	2	3	2	0

copyright©2026 m.patrignani

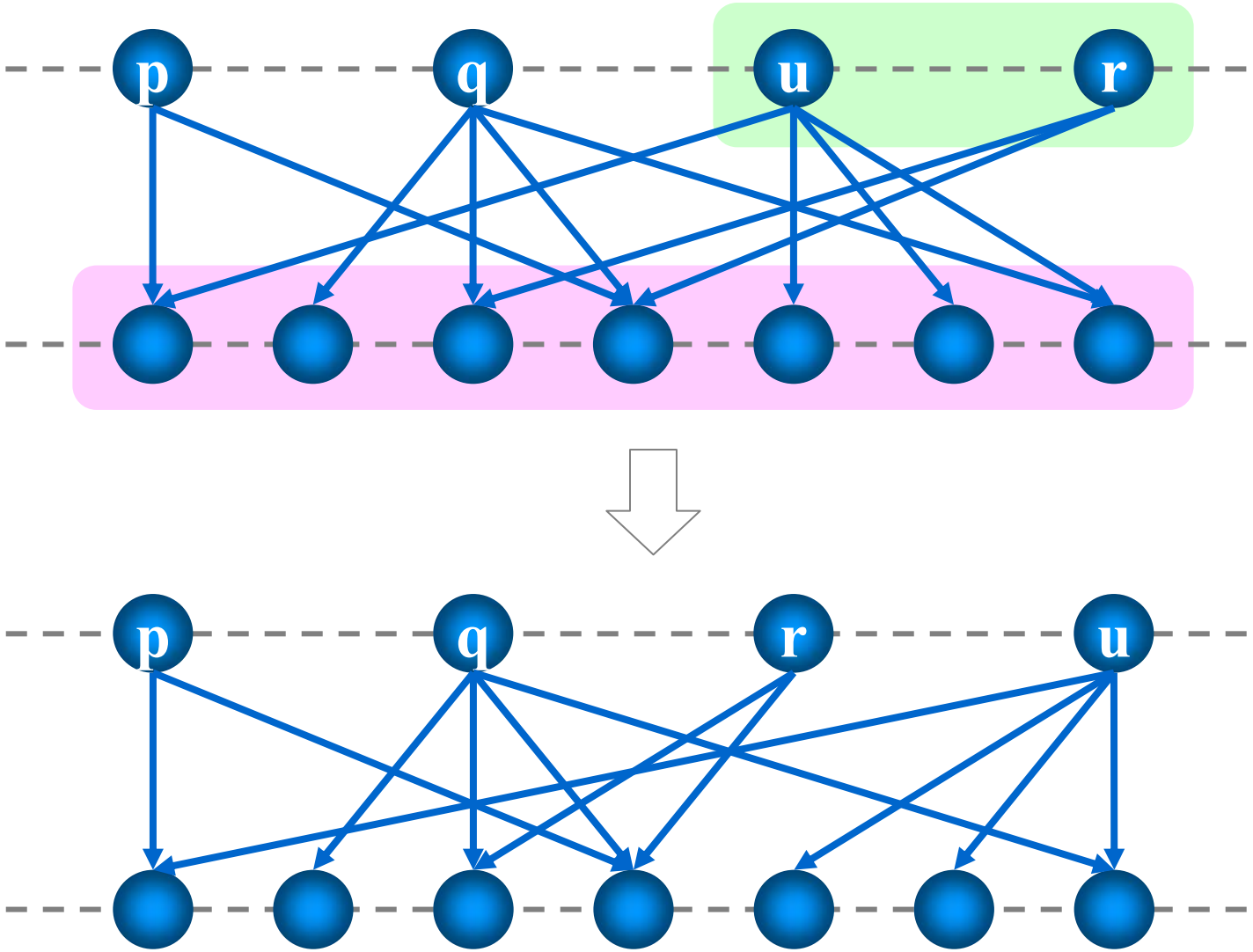
Adjacent-exchange: example



	p	*q*	*u*	*r*
p	0	2	1	1
q	5	0	6	3
u	6	9	0	6
r	2	3	2	0

copyright©2026 m.patrignani

Adjacent-exchange: example



	p	*q*	*u*	*r*
p	0	2	1	1
q	5	0	6	3
u	6	9	0	6
r	2	3	2	0

copyright©2026 m.patrignani

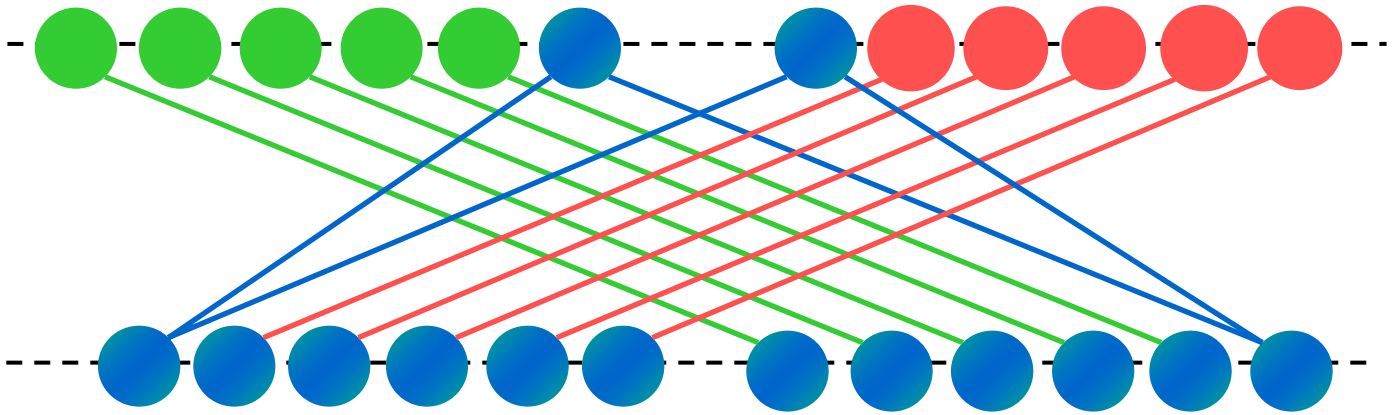
Adjacent-exchange: analysis

- The computation of pair crossing numbers can be performed in $O(|E|^2)$ time
  - $O(\deg(u) \times \deg(v))$ for c_{uv} and c_{vu}
- Since c_{uv} and c_{vu} depend on the relative position of u and v only, it is not needed to recompute the values of pair crossing numbers when a swap is performed
- Each sweep of the nodes may be implemented to run in $O(|L_2|)$ time
- Since there are $O(|L_2|)$ sweeps, the total complexity of step 3 is $O(|L_2|^2)$
- The total complexity is dominated by $O(|E|^2)$

 copyright©2026 m.patrignani

Adjacent-exchange suboptimality

- This instance has $O(|E|^2)$ crossings

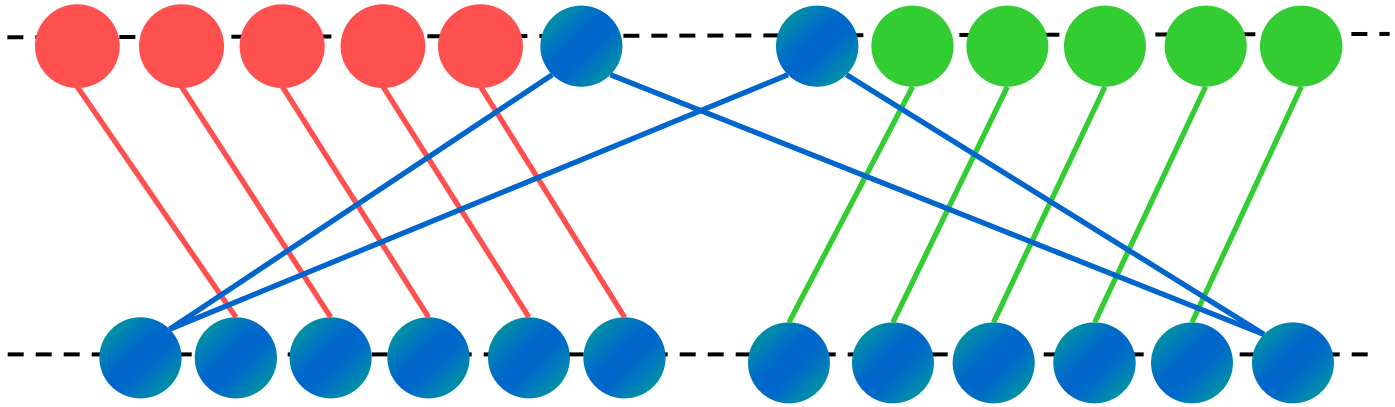


- No swap is performed

 copyright©2026 m.patrignani

Adjacent-exchange suboptimality

- The optimal solution has O(|E|) crossings



- Adjacent-exchange may produces an ordering π_2 that is linearly far from optimum

 copyright©2026 m.patrignani

Split heuristic

- Introduced by [Eades & Kelly, 86]
- Uses an approach analogous to quick-sort
  - choose a pivot node $p \in L_2$
  - place each nodes $u \neq p \in L_2$ to the left of p, if $c_{up} < c_{pu}$, or to the right of p otherwise
  - Recursively apply the procedure to the set of nodes on the left and on the right of p

 copyright©2026 m.patrignani

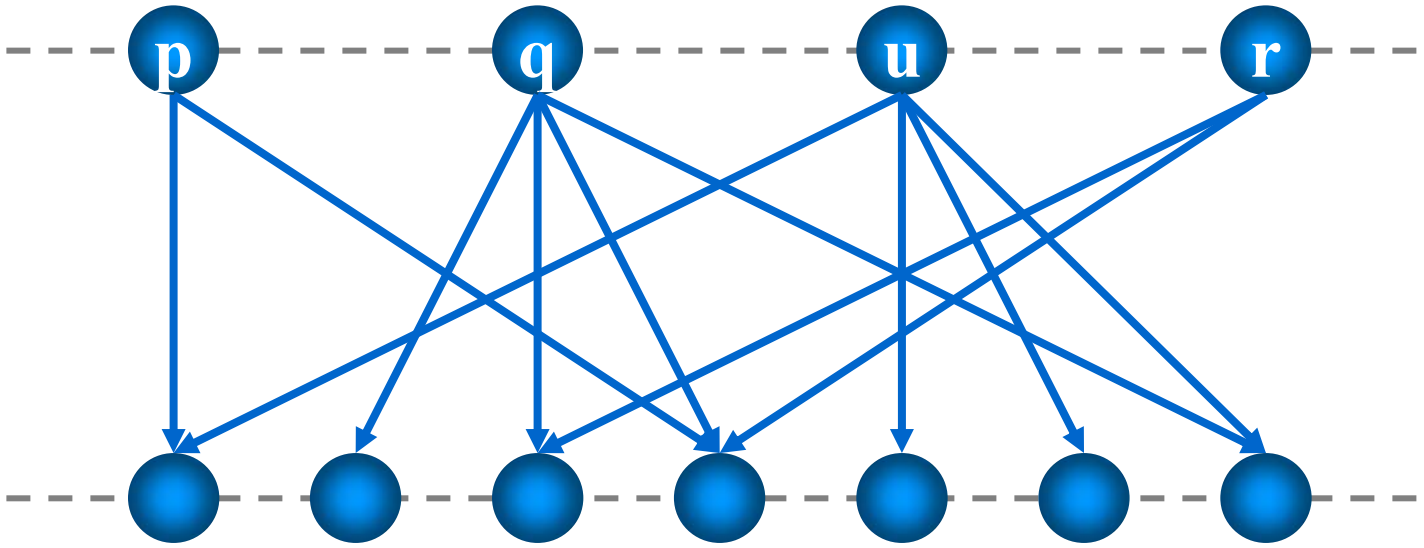
Split heuristic

```
Split_heuristic
/* Input: a two-layered digraph G = (L1, L2, E) and
          an ordering π1 of nodes of L1*/
/* Output: an ordering π2 of nodes of L2 */

    If L2 is not empty
        (a) Choose a pivot node p ∈ L2
        (b) Vleft = ∅; Vright = ∅
        (c) Foreach node u ∈ L2 such that u ≠ p do
                If c_up ≤ c_pu
                Then add u to Vleft
                Else add u to Vright
        (d)Recursively apply the algorithm to the digraphs
           induced by Vleft and Vright and produce as output the
           concatenation of Vleft, p, and Vright
```

 copyright©2026 m.patrignani

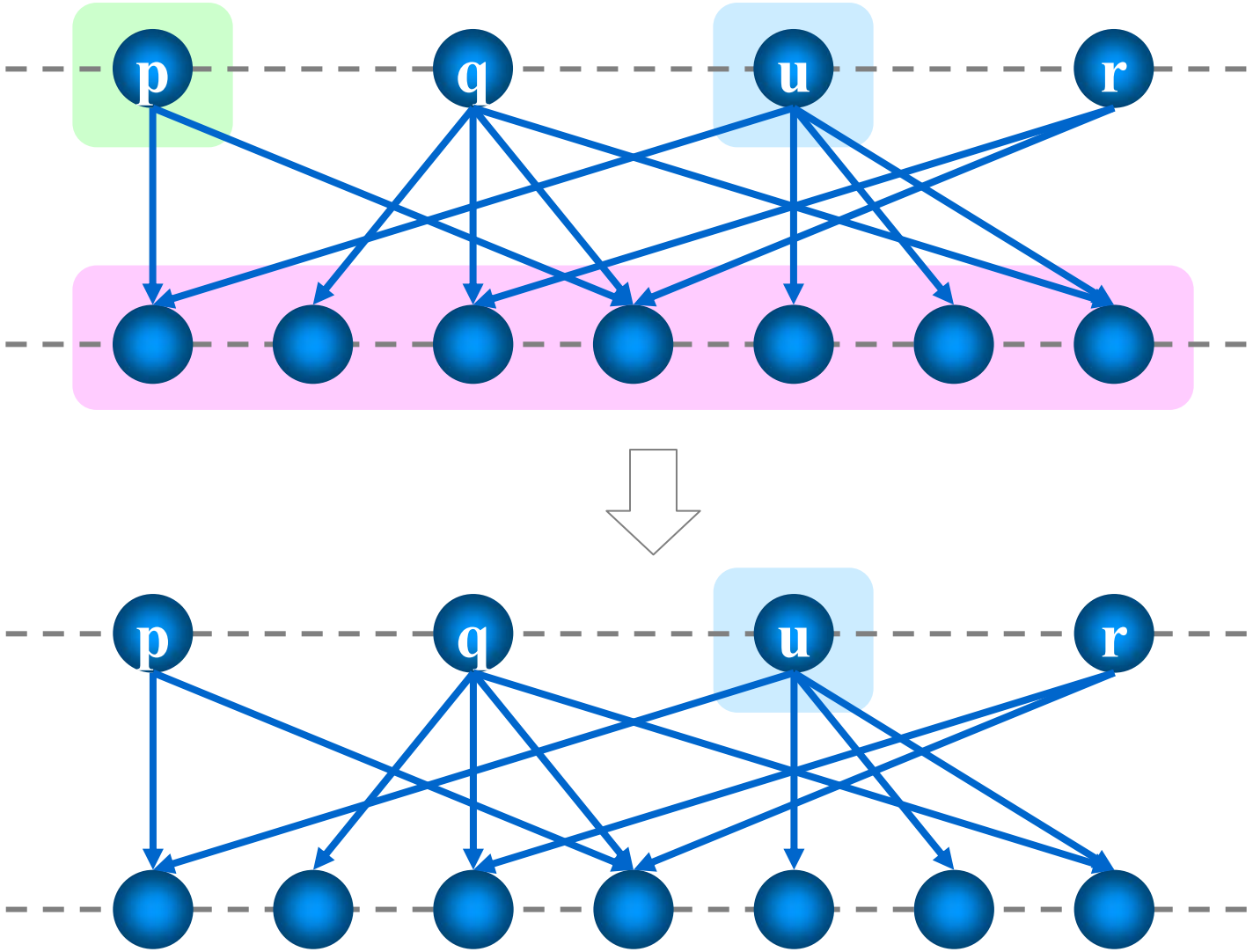
Split heuristic: example



	p	*q*	*u*	*r*
p	0	2	1	1
q	5	0	6	3
u	6	9	0	6
r	2	3	2	0

copyright©2026 m.patrignani

Split heuristic: example



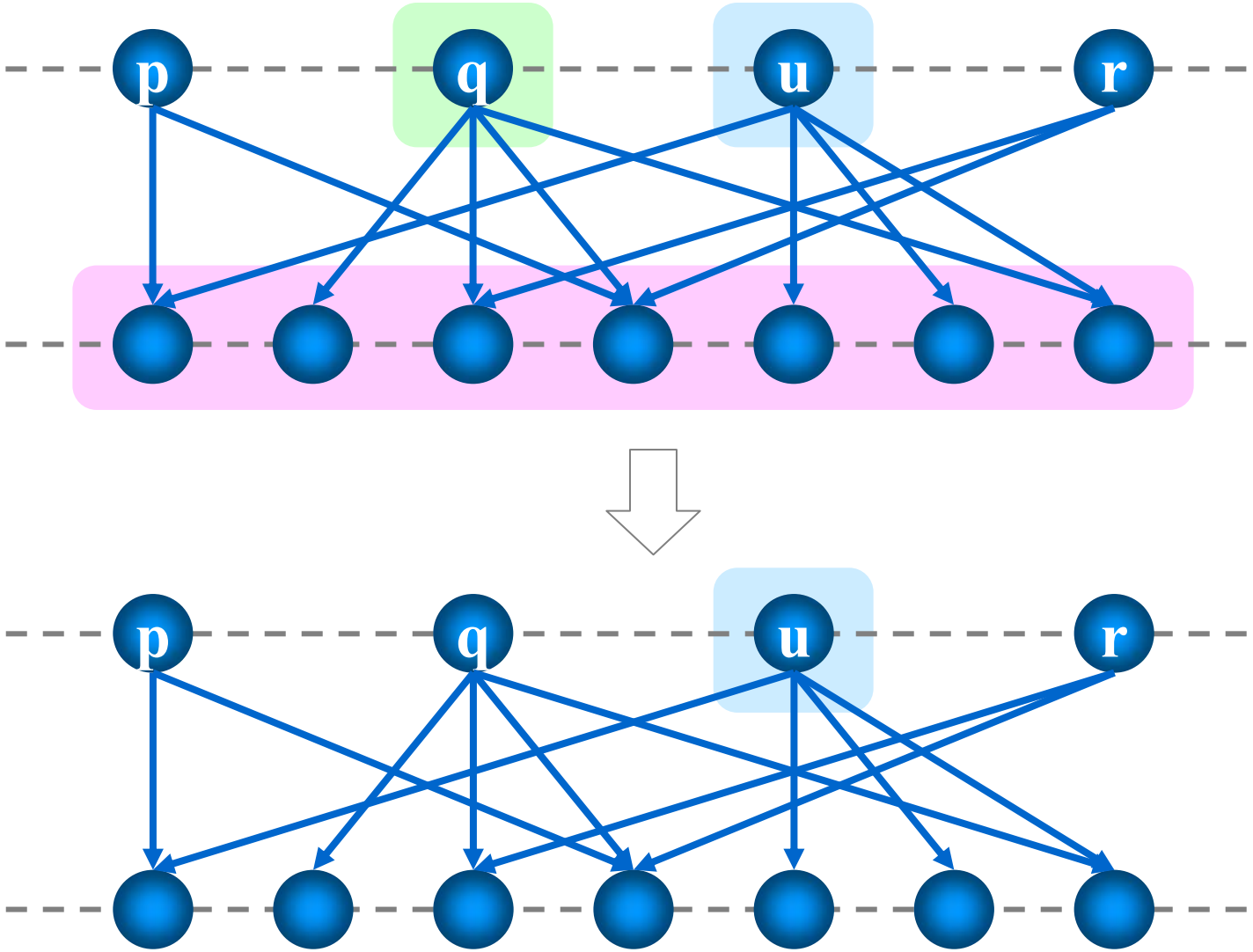
pivot vertex

current vertex

	p	*q*	*u*	*r*
p	0	2	1	1
q	5	0	6	3
u	6	9	0	6
r	2	3	2	0

copyright©2026 m.patrignani

Split heuristic: example



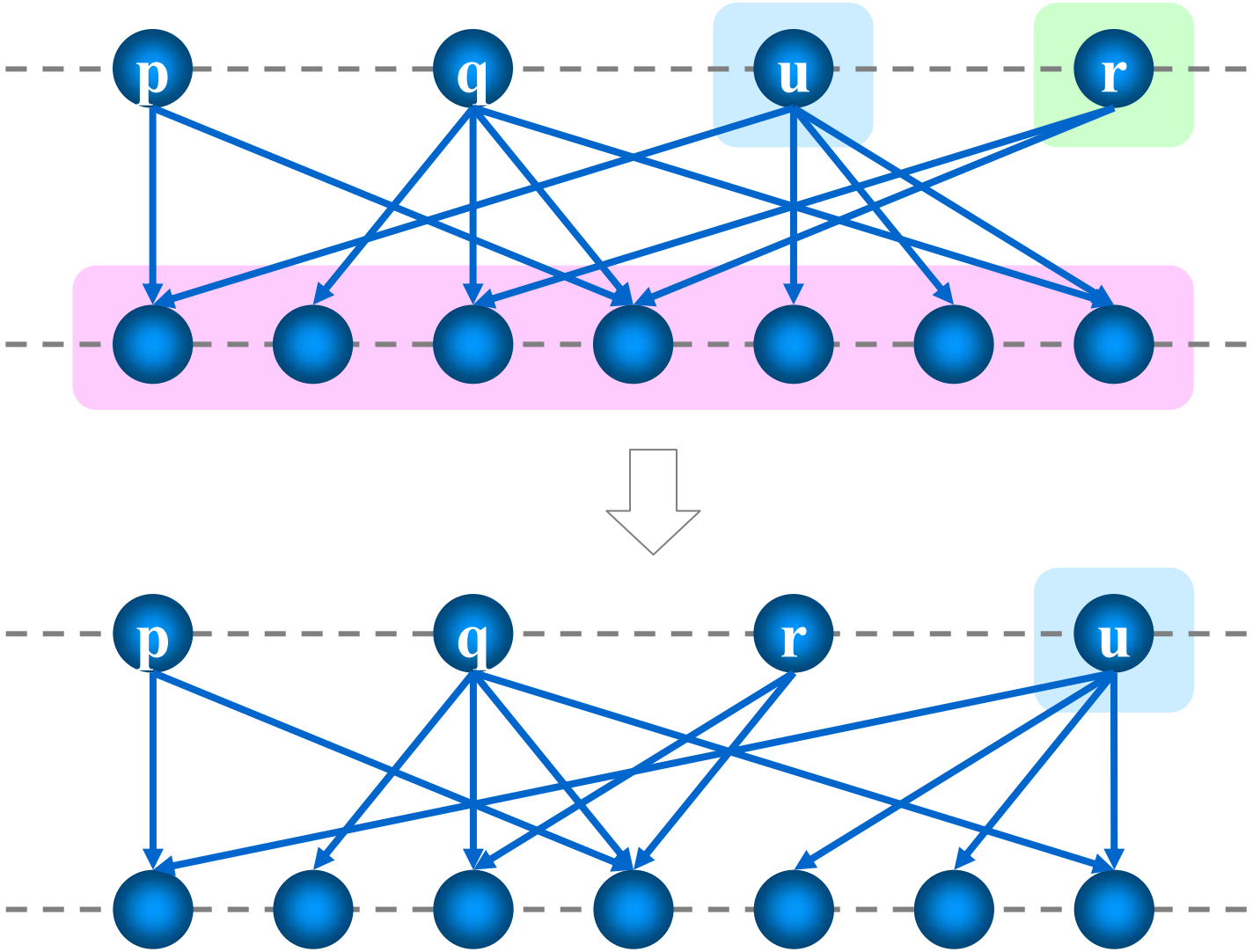
pivot vertex

current vertex

	p	*q*	*u*	*r*
p	0	2	1	1
q	5	0	6	3
u	6	9	0	6
r	2	3	2	0

copyright©2026 m.patrignani

Split heuristic: example



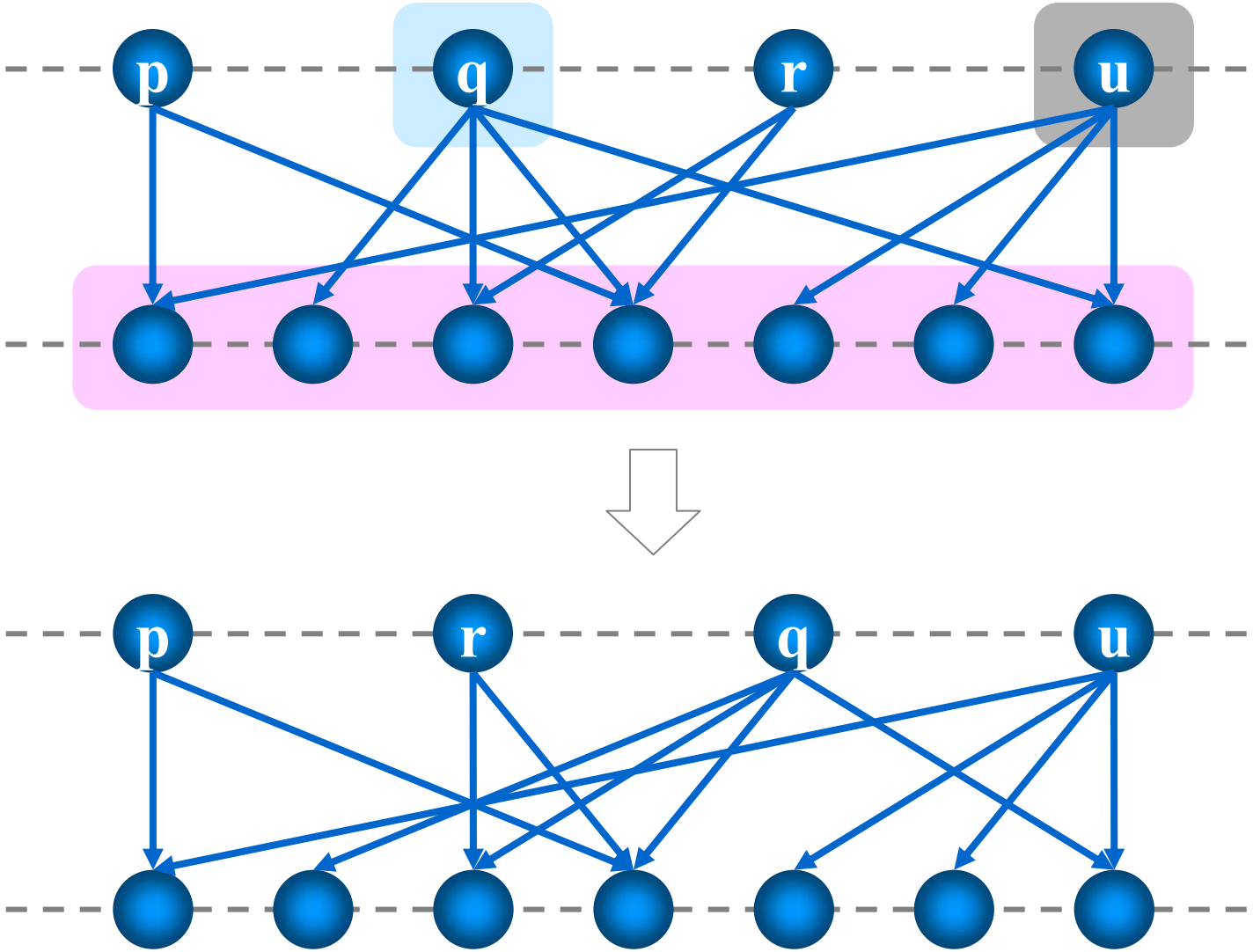
pivot vertex

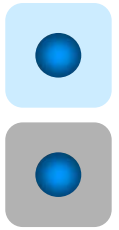
current vertex

	p	*q*	*u*	*r*
p	0	2	1	1
q	5	0	6	3
u	6	9	0	6
r	2	3	2	0

copyright©2026 m.patrignani

Split heuristic: example





	p	*q*	*u*	*r*
p	0	2	1	1
q	5	0	6	3
u	6	9	0	6
r	2	3	2	0

copyright©2026 m.patrignani

Split heuristic: analysis

- The split heuristic has $O(|L_2|^2)$ time complexity in the worst case
  - as for the case of quick sort, experimental analysis shows that its complexity is $O(|L_2|\log|L_2|)$ in several cases
- However, a pre-processing is needed to compute crossing numbers
  - hence, the total complexity is $O(|E|^2)$

 copyright©2026 m.patrignani

Barycenter heuristic

- Introduced by [Sugiyama *et al*., 81]
- The barycenter heuristic is based on the intuition that, in a drawing with few crossings, each node should be near its adjacent nodes
- The barycenter heuristic is
  - easy to implement
  - quick
    - can be implemented to run in linear time in the number of edges
  - usually produces good layouts

 copyright©2026 m.patrignani

Barycenter heuristic

- The position of each node *u* is set as the barycenter (average) of the x-coordinates of the neighbors of u in L_1
  - that is, we assign π_2(u) based on avg(u)

$$avg(u) = \frac{1}{\deg(u)} \sum_{v \in N_u} x(v)$$

  - where
    - deg(u) denotes the degree of u
    - N_u denotes the set of the neighbors of u in L_1

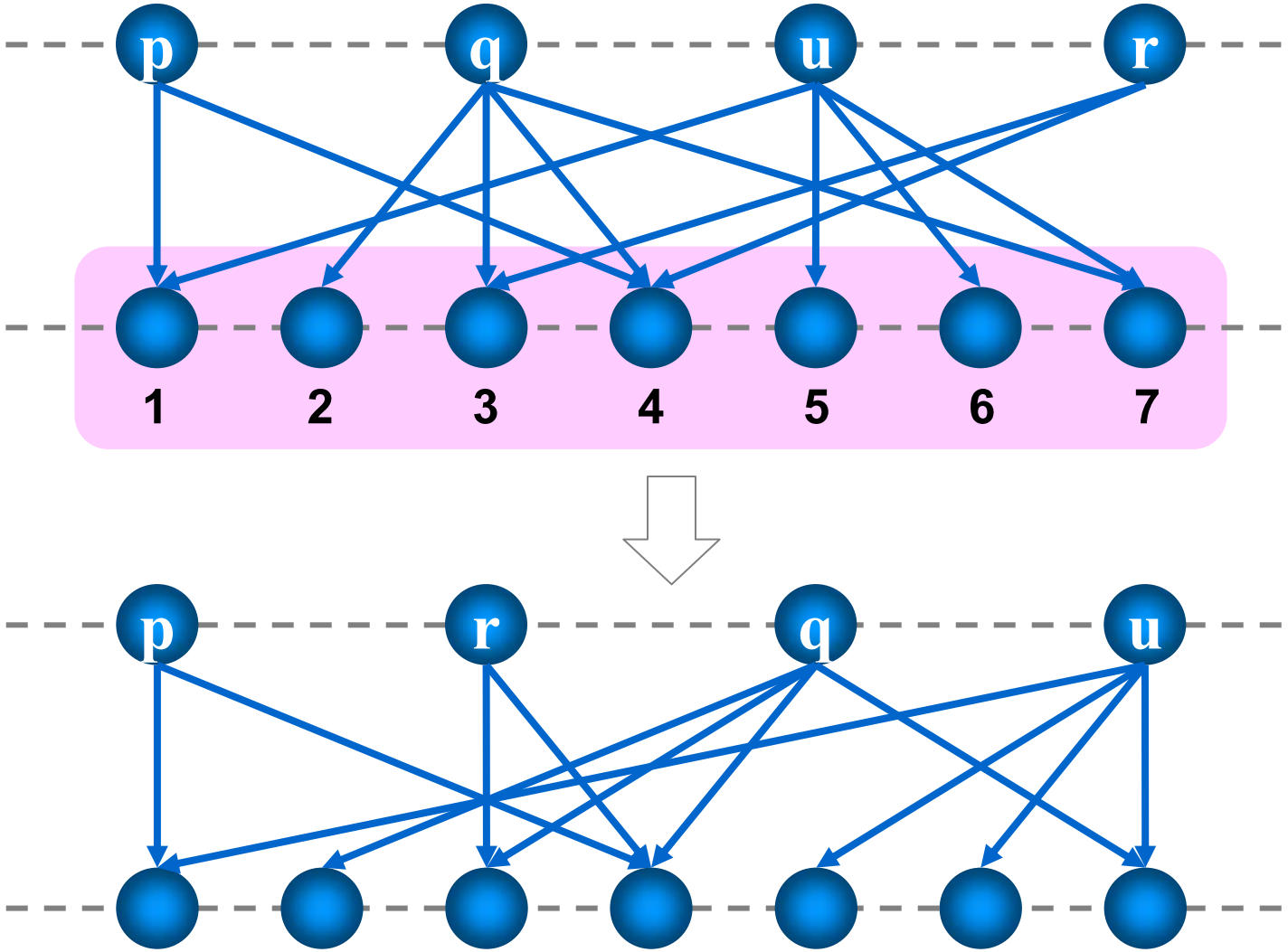
 copyright©2026 m.patrignani

Barycenter heuristic

- Temporarily assign to each node *v* of L_1 an x-coordinate x(*v*) based on the permutation π_1
- For each node *u* of L_2 compute avg(*u*)
- Order nodes of L_2 based on the values of function avg(), namely
  - if avg(*u*) < avg(*v*) then $\pi_2(u) < \pi_2(v)$
  - if avg(*u*) = avg (*v*), i.e., *u* and *v* have the same barycenter, arbitrarily choose their order

 copyright©2026 m.patrignani

Barycenter heuristic: example



$$avg(u) = \frac{1}{\deg(u)} \sum_{v \in N_u} x(v)$$

	avg
p	2.5
q	4
u	5
r	3.5

copyright©2026 m.patrignani

Barycenter heuristic: analysis

- As the time needed to compute avg(*u*) is proportional to the degree of *u*, the barycenters of all the vertices can be computed in time O(|E|)
- Hence, since a sorting algorithm is needed, the total complexity is $O(|L_2|\log|L_2| + |E|)$
  - observe that it is not needed compute crossing numbers, which are instead needed by other heuristics

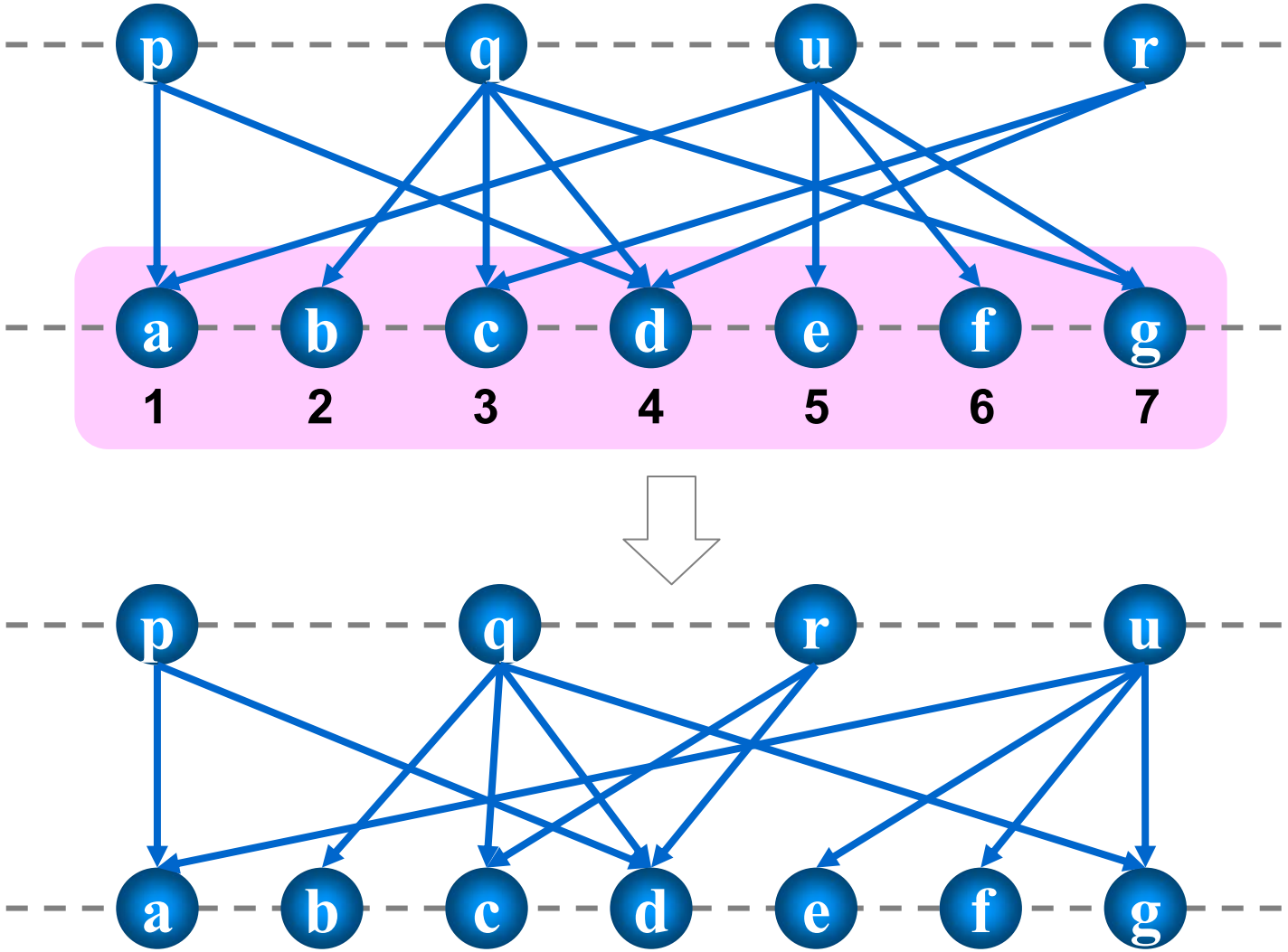
 copyright©2026 m.patrignani

Median heuristic

- Introduced by [Eades & Wormald, 94]
- Similar to the barycenter heuristic
- The x-coordinate of each node *u* of L_2 is chosen as the median of the x-coodinates of the neighbors of *u* in L_1
- The median is defined as follows
  - suppose u has j neighbors $v_0, v_1, \ldots, v_{j-1}$
    - assume $x(v_0) < x(v_1) < \ldots < x(v_{j-1})$
  - set $med(u) = x(v_{\lfloor j/2 \rfloor})$
- Order nodes of L_2 based on their med()
  - if $med(u) = med(v)$ and *v* has even degree while *u* has odd degree, then $\pi_2(v) < \pi_2(u)$
  - if $med(u) = med(v)$ and both the nodes have even or odd degree, choose arbitrarily the order of *u* and *v*

 copyright©2026 m.patrignani

Median heuristic: example



$$med(u) = x(v_{\lfloor j/2 \rfloor})$$

	med	*deg*
p	$x(d)=4$	2
q	$x(d)=4$	4
u	$x(f)=6$	4
r	$x(d)=4$	2

from 17 to 12 crossings

copyright©2026 m.patrignani

Median heuristic: analysis

- For each node $u \in L_2$, med(u) can be computed in a time proportional to the degree of u
- The value med(u) is an integer between 1 and $|L_1|$
  - hence, they can be sorted in $O(|L_1|+|L_2|)$
- Overall, the ordering of L_2 can be computed in $O(|L_1| + |L_2| + |E|)$

 copyright©2026 m.patrignani

Observations

- It is easy to prove that the median heuristic and the barycenter heuristic bring to a drawing without crossings if the graph admits one
- None of the two provides, in general, an optimal solution
- For the median heuristic it was proved that

$$med\ (G, \pi_1) \leq 3\, opt\ (G, \pi_1)$$

 copyright©2026 m.patrignani

Final remarks on crossing reduction

- From a theoretic point of view the median heuristic is preferable
  - it computes an ordering that approximate the optimal within a constant factor
  - it can be computed in O(|V|+|E|) time
- Actually, extensive experimental tests show that there is no heuristic that can be preferred over the others
  - the best practical approach seems to be an hybrid approach that uses first the median heuristic and then refines the solution with adjacent-exchange or split heuristic

 copyright©2026 m.patrignani

Step 4: X-coordinate assignment

 copyright©2026 m.patrignani

Problem statement

Input	A proper layering of a directed acyclic graph G and an ordering of the nodes on each level
Output	An x-coordinate value v(x) for each node v
Target	Reduce the number of bends along the edges

- Recall that bends occur in correspondence to dummy nodes only

 copyright©2026 m.patrignani

Approaches to the problem

- Usually this problem is solved by formulating it as a mathematical programming problem and then launching a mathematical solver
- A subset of the following aesthetics are taken into account
  - alignment of dummy nodes to avoid bends
  - closeness to connected vertices
  - centering between predecessors and successors

 copyright©2026 m.patrignani

An optimization formulation

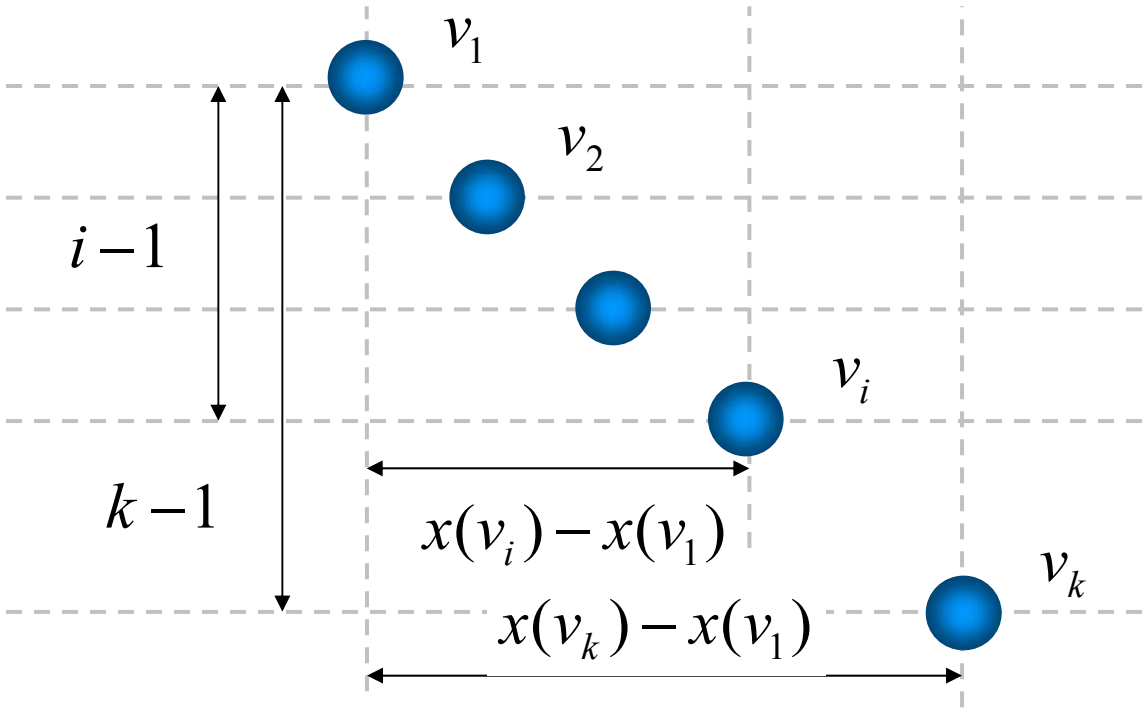
- X-coordinate assignment can be formulated as an optimization problem with a quadratic objective function as follows
  - consider the directed path p = (v_1, v_2, …, v_{k-1}, v_k), where v_2, …, v_{k-1} are dummy nodes
  - if there is no bend, then all vertices of p are collinear

 copyright©2026 m.patrignani

An optimization formulation

- In order to be collinear each v_i ($2 \le i \le k-1$) should have x-coordinate $x(v_i)$ such that:

$$x(v_i) - x(v_1) = \frac{i-1}{k-1}(x(v_k) - x(v_1))$$



 copyright©2026 m.patrignani

An optimization formulation

$$x(v_i) - x(v_1) = \frac{i-1}{k-1}(x(v_k) - x(v_1))$$

- the ideal value for $x(v_i)$ is

$$a_i = \frac{i-1}{k-1}(x(v_k) - x(v_1)) + x(v_1)$$

- for each directed path *p*, starting at v_1 and ending at v_k with all internal dummy nodes, we define

$$g(p) = \sum_{i=2}^{k-1} (x(v_i) - a_i)^2$$

 copyright©2026 m.patrignani

An optimization formulation

- In order to minimize the bends along edges, we need to minimize the sum

$$\sum_{p} g(p)$$

- on all paths of dummy nodes, with the constraint that

$$x(w) - x(z) \geq \delta$$

- for all pairs *w*, *z* of nodes on the same layer, with *w* on the right of *z*

 copyright©2026 m.patrignani

Final considerations

- Drawing hierarchical graphs is a fascinating problem
- Huge literature
  - NP-hard problems
  - intuitive heuristics
  - still room for good intuitions
- Simple algorithms to produce hierarchical drawings are easy to implement

 copyright©2026 m.patrignani

Bibliography and further readings

- [Di Battista *et al.*, 99] G. Di Battista, P. Eades, R. Tamassia, and I. G. Tollis, “Graph Drawing: Algorithms for the Visualization of Graphs”, Prentice-Hall, 1999
- [Eades & Kelly, 86] P. Eades and D. Kelly, “Heuristics for reducing crossings in 2-layered networks”, Ars Combinatoria, 1986
- [Eades & Whitesides, 94] Peter Eades and Sue Whitesides “Drawing Graphs in Two Layers”, Theor. Comput. Sci. 131, 1994
- [Eades & Wormald, 94] P. Eades and N. C. Wormald, “Edge crossings in drawings of bipartite graphs”. Algorithmica, 1994.
- [Shaefer 13] Marcus Schaefer. “The graph crossing number and its variants: A survey”. Electronic Journal of Combinatorics, #DS21, 2013.
- [Sugiyama *et al*., 81] Kozo Sugiyama, Shojiro Tagawa, Mitsuhiko Toda, “Methods for Visual Understanding of Hierarchical System Structures” IEEE Trans. on Systems, Man, and Cybernetics,1981

 copyright©2026 m.patrignani